

Holiday Study Buddy: A Gamified Productivity Platform

Technical Report & Architecture Overview

Development Team

December 16, 2025

Abstract

This report details the technical architecture and implementation of **Holiday Study Buddy**, a gamified productivity application designed to combat holiday procrastination. Beyond standard task management, the system integrates a robust virtual economy, role-playing elements (XP, Levels), and a secure backend infrastructure. Special emphasis is placed on the implementation of enterprise-grade security measures, including **prevention of race conditions in virtual currency transactions**, **application-level Denial of Service (DoS) mitigation**, and **stateless JWT authentication**.

Contents

1	Introduction	2
2	System Architecture	2
3	Technical Highlights & Security Engineering	2
3.1	Concurrency Control: Solving the "Double-Spending" Problem	2
3.2	Resilience: Application-Level Rate Limiting	3
3.3	Security: Stateless Authentication with JWT	3
4	Core Features	4
5	Conclusion	4

1 Introduction

The "Holiday Study Buddy" gamifies the productivity experience by transforming mundane tasks into RPG-style quests. Users earn Experience Points (XP) and virtual currency by completing tasks and maintaining focus sessions. This report highlights the engineering challenges solved to ensure the integrity of this virtual economy and the security of user data.

2 System Architecture

The application follows a modern client-server architecture:

- **Frontend:** Built with **Next.js (React)**, utilizing server-side rendering for performance and a responsive UI/UX.
- **Backend:** A RESTful API developed with **Python (Flask)**, chosen for its flexibility and rapid development capabilities.
- **Database:** **SQLite** (with SQLAlchemy ORM) for reliable, relational data persistence.

3 Technical Highlights & Security Engineering

This section details the advanced engineering implementations that distinguish Holiday Study Buddy from typical CRUD applications.

3.1 Concurrency Control: Solving the "Double-Spending" Problem

In any system with a virtual currency, a critical vulnerability is the *Race Condition*. A malicious user could exploit network latency to send multiple "Purchase" requests simultaneously. Without proper locking, the server might process all requests against the same initial balance, allowing the user to buy multiple items while paying for only one (Double-Spending).

Our Solution: Atomic Transactions with Row Locking We implemented pessimistic concurrency control using database row locking. When a user attempts to redeem a gift:

1. A database transaction is initiated.

2. The user's record is locked using `SELECT ... FOR UPDATE` (via SQLAlchemy's `with_for_update()`).
3. Other transactions attempting to modify this user's balance are forced to wait until the lock is released.
4. The system verifies the balance, deducts the cost, and grants the item atomically.

```
1 # Atomic Transaction with Row Locking
2 try:
3     # Lock the user row for update
4     user = db.session.query(User).with_for_update().get(user_id)
5
6     # ... Verify Balance and Item Availability ...
7
8     # Deduct XP and Unlock
9     user.xp -= gift.xp_required
10    db.session.add(new_user_gift)
11    db.session.commit() # Lock released here
12 except Exception:
13    db.session.rollback()
```

Listing 1: Atomic Transaction Implementation

3.2 Resilience: Application-Level Rate Limiting

To protect the system availability and database integrity, we identified a potential Denial of Service (DoS) vector where a user could flood the database with an infinite number of tasks.

Our Solution: Resource Quotas We implemented application-level logic to enforce strict quotas on active resources. Before creating a new task, the system checks the user's current active load.

- **Mechanism:** A pre-flight check query counts active tasks for the user.
- **Policy:** Users are limited to 50 active tasks.
- **Impact:** This prevents database bloat and ensures query performance remains optimal for all users, effectively mitigating resource exhaustion attacks.

3.3 Security: Stateless Authentication with JWT

Moving beyond simple session-based login, we implemented a stateless authentication system using **JSON Web Tokens (JWT)**.

- **Token Generation:** Upon login, the server signs a token containing the user's identity and an expiration time using a cryptographically secure secret key.

- **Request Validation:** A custom decorator `@token_required` intercepts every protected request. It validates the token signature and expiration before allowing access to the endpoint.
- **Security Hardening:** The system uses `os.urandom(24)` to generate high-entropy secret keys, preventing brute-force attacks on the token signature.

4 Core Features

- **Focus Sessions:** Server-side validation ensures that reported focus duration matches the actual time elapsed, preventing client-side clock manipulation exploits.
- **Quest System:** An automated progression system that tracks user activities and awards badges/XP based on complex criteria (e.g., "Task Master", "Dedicated Student").
- **Virtual Store:** A secure marketplace where users can redeem earned XP for virtual gifts, powered by the atomic transaction system described above.

5 Conclusion

Holiday Study Buddy demonstrates that a fun, gamified user experience can be built on a foundation of rigorous software engineering. By proactively addressing concurrency issues, resource exhaustion, and authentication security, the application provides a robust and secure platform for productivity.